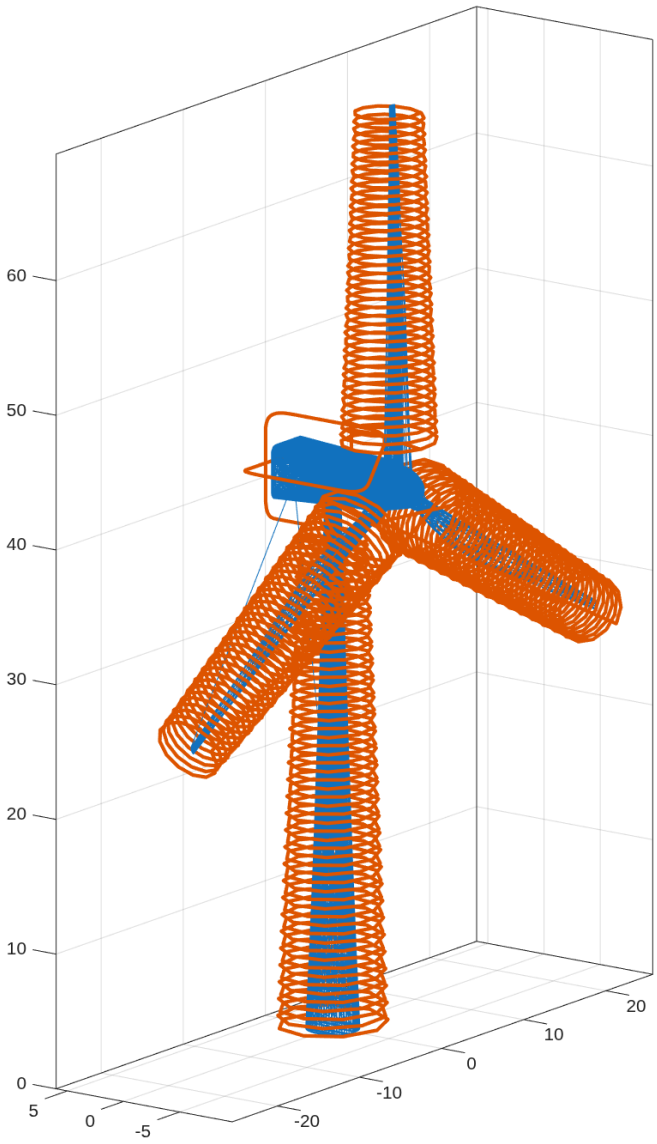
R7010E Lab 1



Gustav Juul, gusjuu-3, juulgustav@gmail.com

LULEÅ TEKNISKA UNIVERSITET

November 13, 2026

1 Objective

1.1 Abstract

This lab was focused on learning about Matlab and Peter Corke's Robotics Toolbox. Specifically we learned to create trajectories from base points and then create combined plots with the trajectories and structures.

1.2 Introduction

Some potential difficulties I believe I might face are technical difficulties with Matlab. Usually when Matlab is uncooperative it's because of simulink. As this lab is 'pure' Matlab this hopefully won't be an issue. I also hope the math in the lab won't be too complicated. I know control theory of course but I also remember how much time and effort those labs took.

2 Task 1

In the first task we create an animated trajectory with a displayed axis between 4 points.

```
via = [B(1:3); C(1:3); D(1:3)];
tseg = [1 1 1];
q0 = A(1:3);
% Sampling time
dt = 0.01;
% Acceleration time
tacc = 0.5;
% Generate trajectory
q = mstraj(via, [], tseg, q0, dt, tacc);
via = [B(4:6); C(4:6); D(4:6)];
q0 = A(4:6);
rpy = mstraj(via, [], tseg, q0, dt, tacc);
plot_points(x,y,z)
plot3(q(:,1), q(:,2), q(:,3))
T = SE3(q) .* SE3.rpy(rpy);
T.animate('length', 0.2);
hold off
```

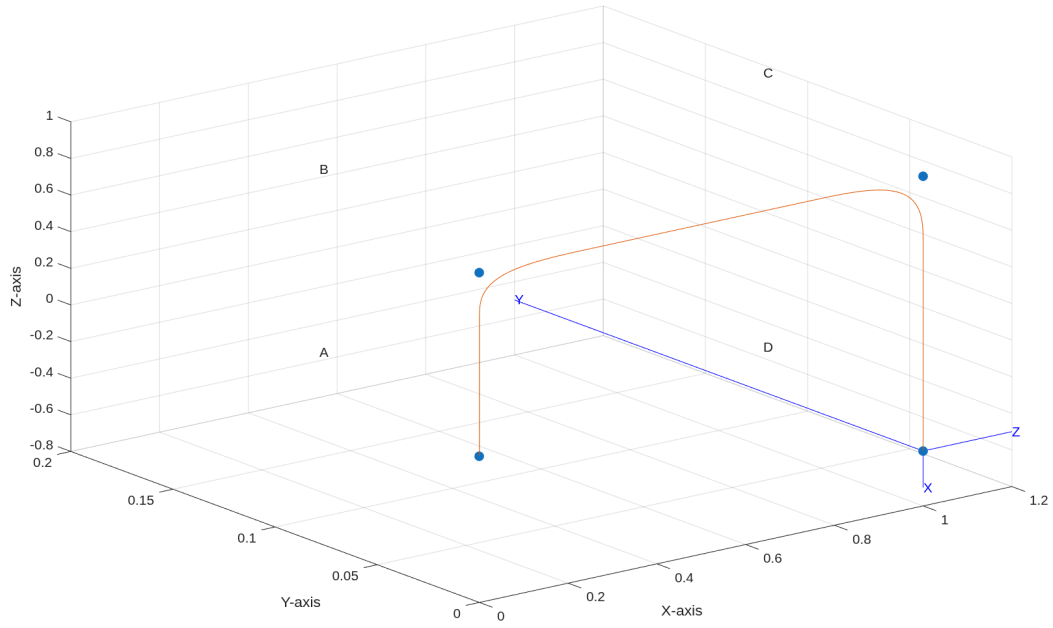


Figure 1: Trajectory and axis plot

3 Task 2

Task 2 was a bit simpler as most of the code was already written for you. All you had to do was supply the points that would be used to generate the trajectory and adjust max velocity. You also had to make you robot avoid hitting the door, most easily achieved by adjusting max velocity.

```
path=[ p_init;...
       15 35 ;...
       15 65 ;...
       50 65 ;...
       50 100 ;...
       90 100 ;...
       90 0 ;...
       p_goal
    ];
```

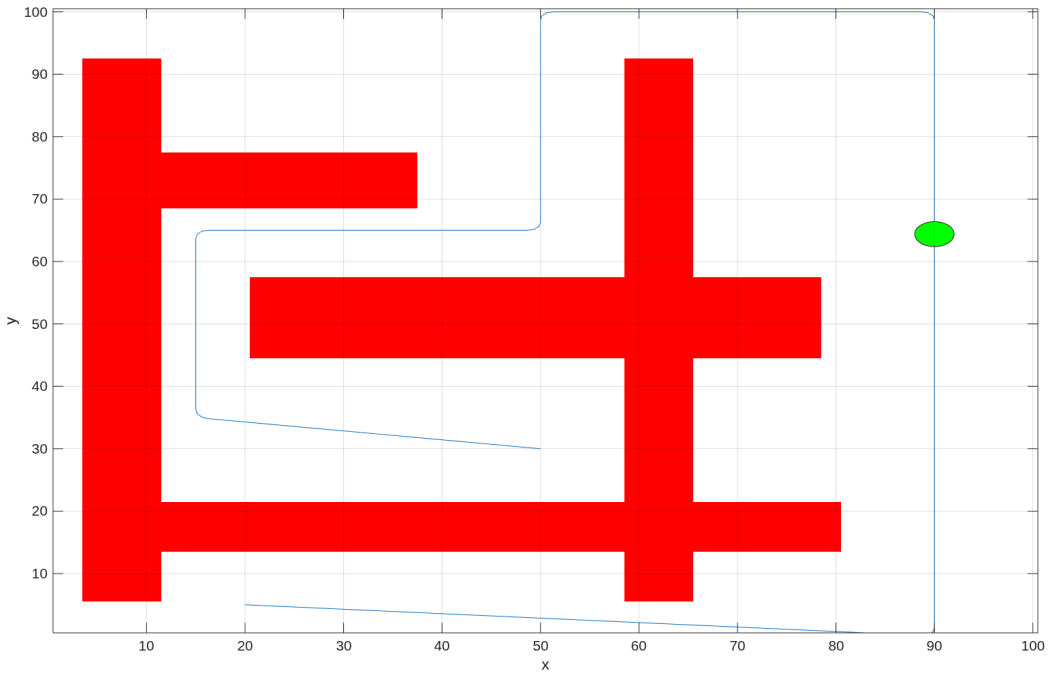


Figure 2: Trajectory plot of ground robot

4 Task 3

In task 3 we used a simulated PUMA560 robot to draw letters on a wall.

4.1 Vertical

For vertical both the first letter of your first name and last name should be drawn in one trajectory where the pen is lifted between characters. The characters should also be written at different speeds. As usual mstraj converts the base points to a trajectory.

```
q_j_end = tail(q_j,1); % q_j is trajectory for j
q_j_end(1,1) = -0.01; % lift pen
q_g_start = head(q_g,1); % q_g is trajectory for
    g
q_g_start(1,1) = -0.01; % lift pen
q_jg = mstraj(q_jg, qdmax, [], [], dt, tacc); %
   trajectory between j and g
traj=[q_j; q_jg; q_g]; % final trajectory
```

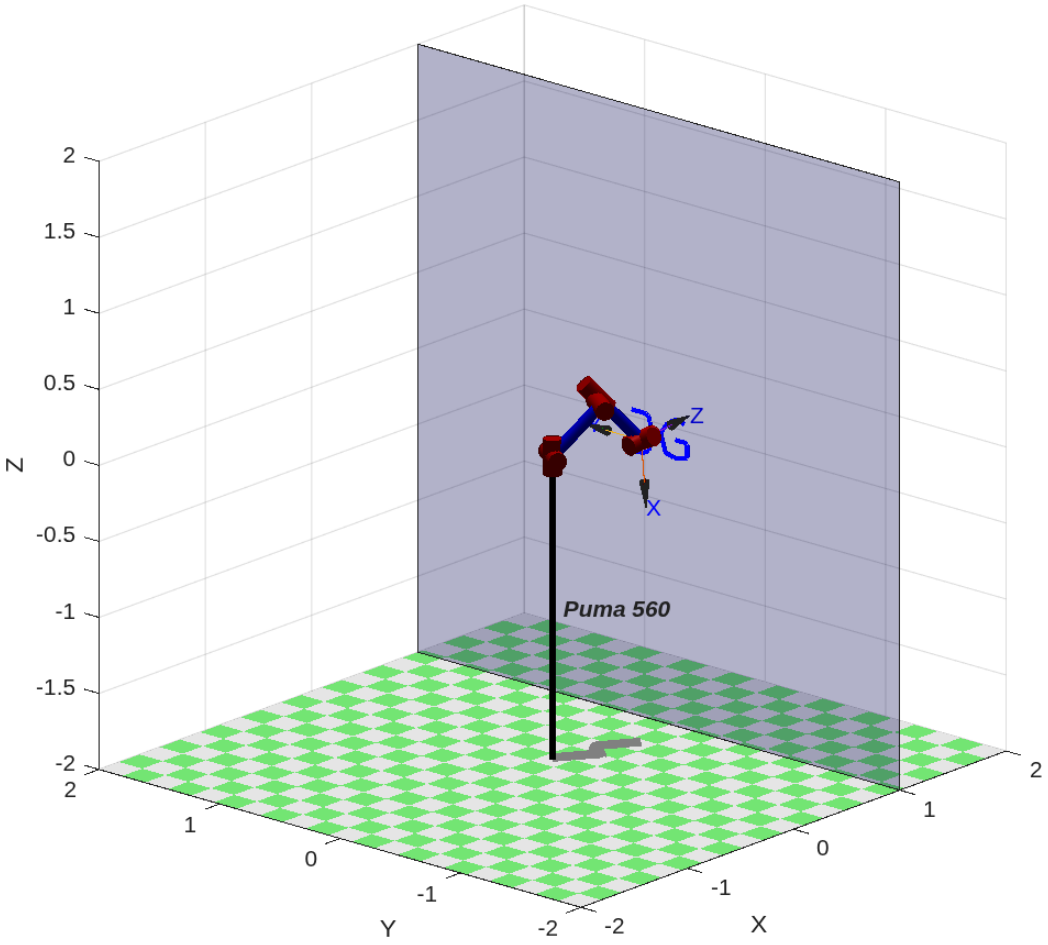


Figure 3: Trajectory plot of vertical PUMA560

4.2 Inclined

For the inclined version only one letter has to be displayed. In order to achieve full contact with the wall each point is offset by $-\frac{c}{a}$. It's not a real rotation but because the incline is so slight it isn't really noticeable so works well in this instance. For steeper inclines a real rotation should be implemented.

```
% Offset depth
k=-c/a; % c and a here defines the slope of the
    wall by ax+by+cz+d = 0
length=size(q(:,1));
for i = 1:length(1)
    q(i,1) = q(i,3)*k;
end
```

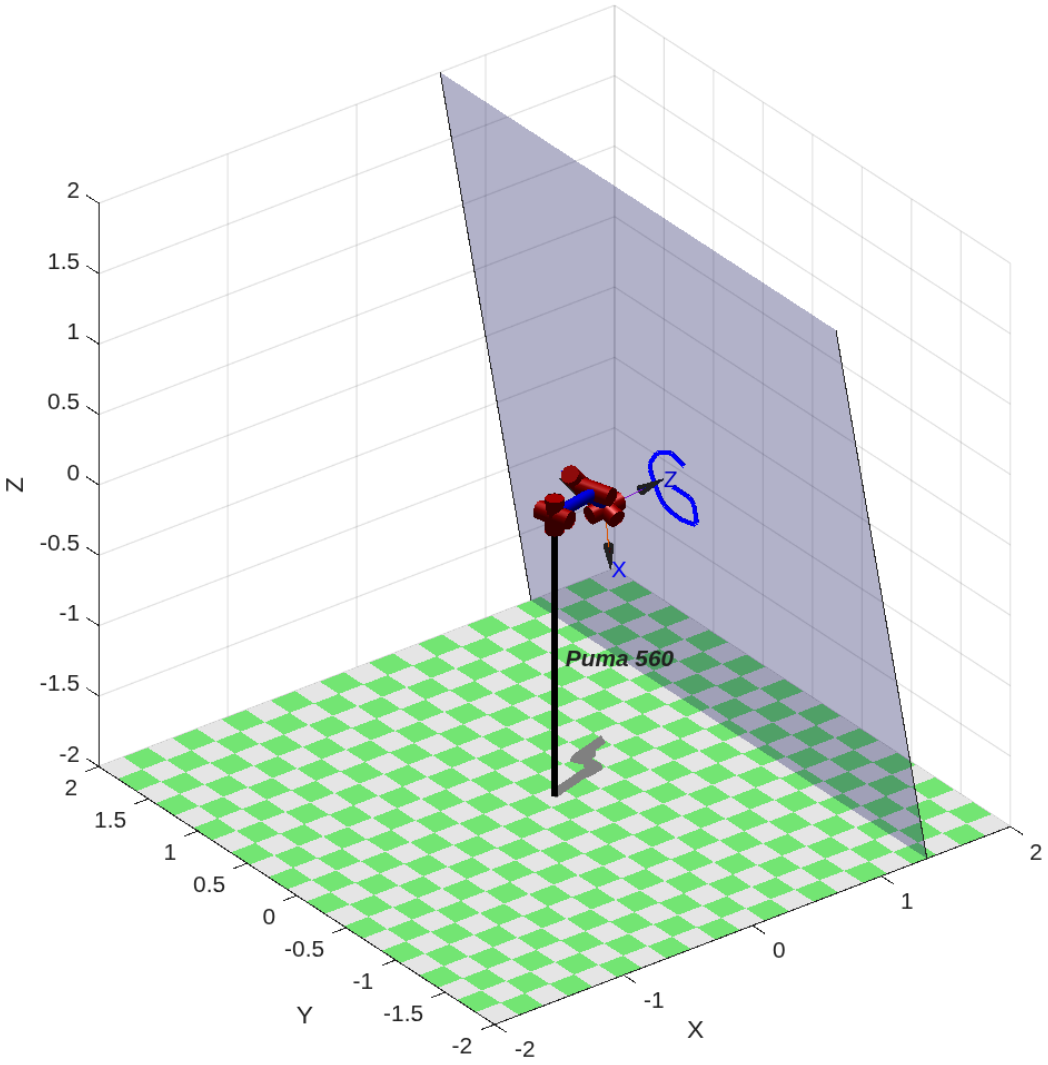


Figure 4: Trajectory plot of inclined PUMA560

5 Task 4

Task 4 was focused on generating trajectories for drones. The first two parts were on navigating through an environment and the other two were on scanning structures while maintaining a safe distance.

5.1 Map 1

For the first map a simple for loop to generate the points worked well. Mstraj then generated a functional trajectory from the points.

```
% Generating base points
points = [-5 2 0];
for i = 1:4
    offset =((i-1)*6);
    points = [points; -1+offset 2 4.5];
    points = [points; 2+offset 2 4.5];
    points = [points; 2+offset 2 1];
    points = [points; 5+offset 2 1];
end
```

```
points(end, :) = [22, 2, 0];
```

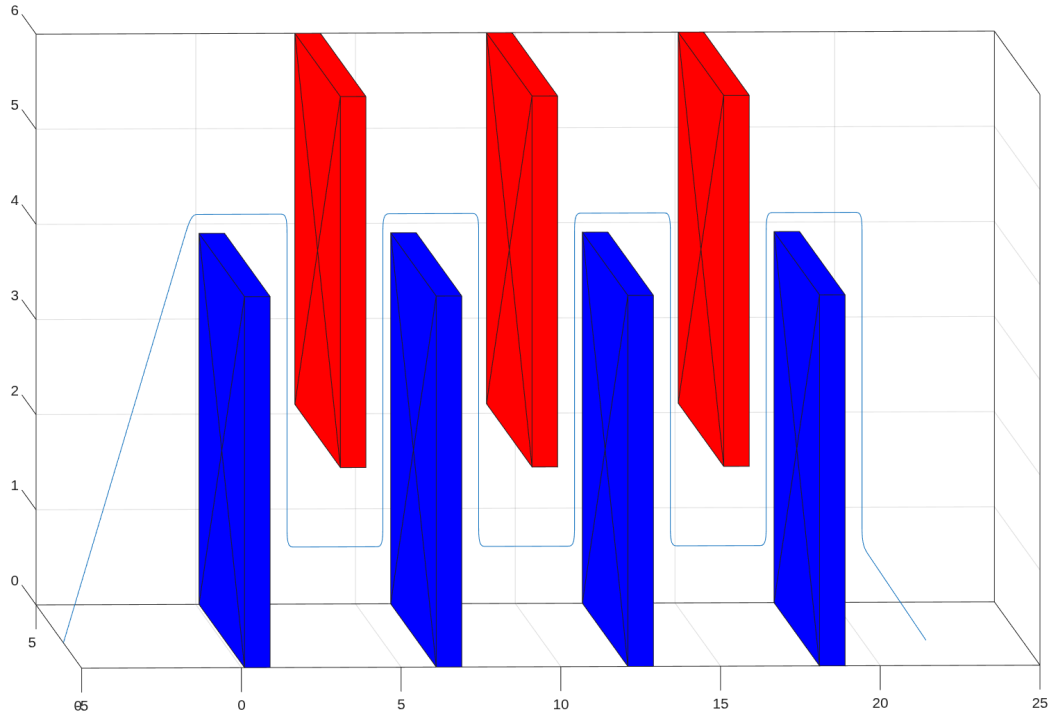


Figure 5: Trajectory plot of drone traversing Map 1

5.2 Map 2

Since map 2 was simpler there was no need for a loop and some hard-coded points instead solved it. Mstraj as usual was used to convert the points into a trajectory.

```
% Hardcoded points
points = [
5 1 0;
5 1 2;
5 14 4;
5 17 4;
5 20 2;];
```

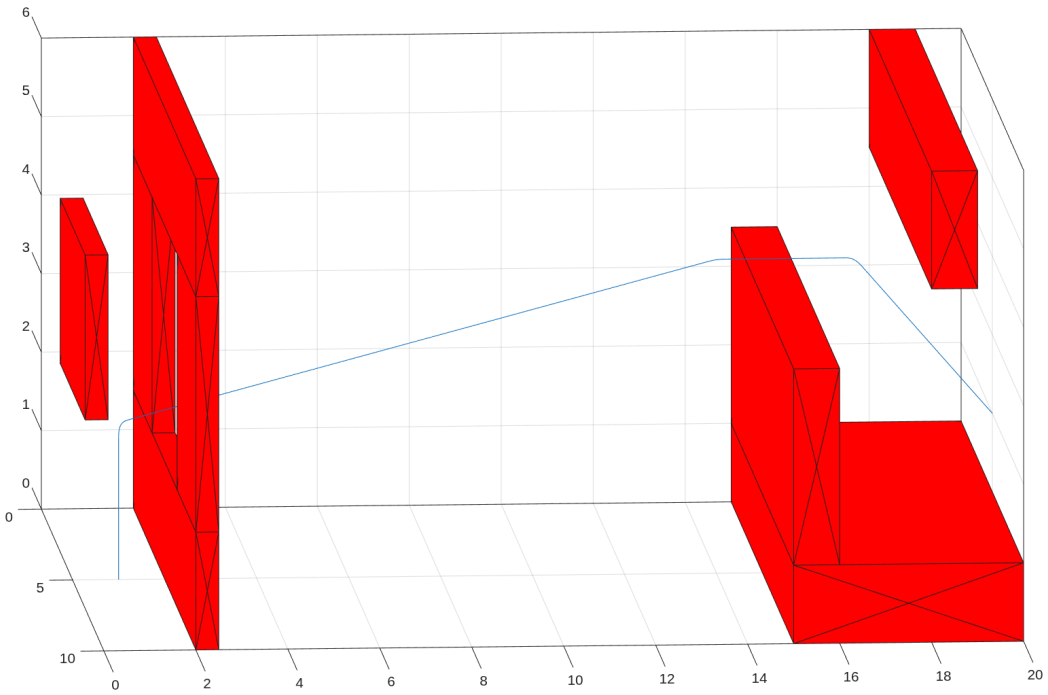


Figure 6: Trajectory plot of drone traversing Map 2

5.3 Fountain

Since the fountain is shaped like a cylinder, a helix plot scans it easily. This function was reused for the wind turbine

```
function path = generate_fountain_path()
    height = 16.5;
    % Spiral radius and vertical growth rate
    r = 5;
    c = 0.1;

    % Define parameter t for the spiral turns
    length = height/(pi*c);
    start= pi*5;
    t = linspace(start, length*pi, 500);

    % Parametric equations
    x = r * cos(t);
    y = r * sin(t);
    z = c * t;
    path = [x' y' z'];
end
```

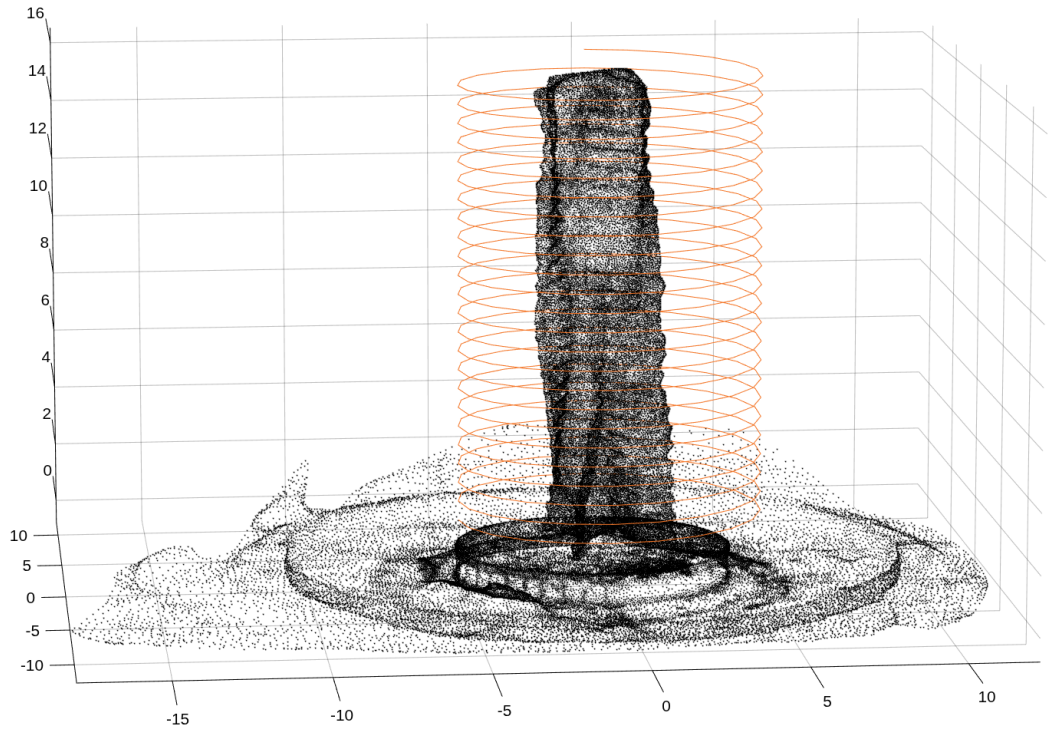


Figure 7: Trajectory plot of drone scanning the LTU fountain

5.4 Windturbine

The wind turbine is similar to the fountain as it's mostly cylinder shaped with the blade being cylindrical protrusions from the hub. Because of this we used four helix path with some manual points in order to scan the hub.

```
% Create segments
r = linspace(4, 2.5, 500);
path = generate_helix_path(37, r, 0.1);
r = linspace(3.5, 2.5, 500);
path2 = generate_helix_path(25, r, 0.1);
path2 = rotate_path(120, path2) +[3 -5 39];
path3 = generate_helix_path(25, r, 0.1);
path3 = rotate_path(-120, path3) +[-3 -5 39];
path4 = [
0 -10 41;
3 -5 41;
3 6 41;
0 6 41;
-3 6 41;
-3 -5 41;
0 -5 45;
0 6 45;
0 6 37;
0 2 37;
5 2 39;
5 2 43];
path4 = mstraj(path4, qdmax, [], tail(path3,1),
    dt, tacc);
path5 = generate_helix_path(25, r, 0.1);
path5 = path5 +[0 -5 44];
```

```
% Connect path segments
path = [path; blend_points(tail(path,1), head(
    path2))];
path2 = [path2; blend_points(tail(path2,1), head
    (path3))];
path3 = [path3; blend_points(tail(path3,1), head
    (path4))];
path4 = [path4; blend_points(tail(path4,1), head
    (path5))];
% Combine
path = [path; path2; path3; path4; path5];
```

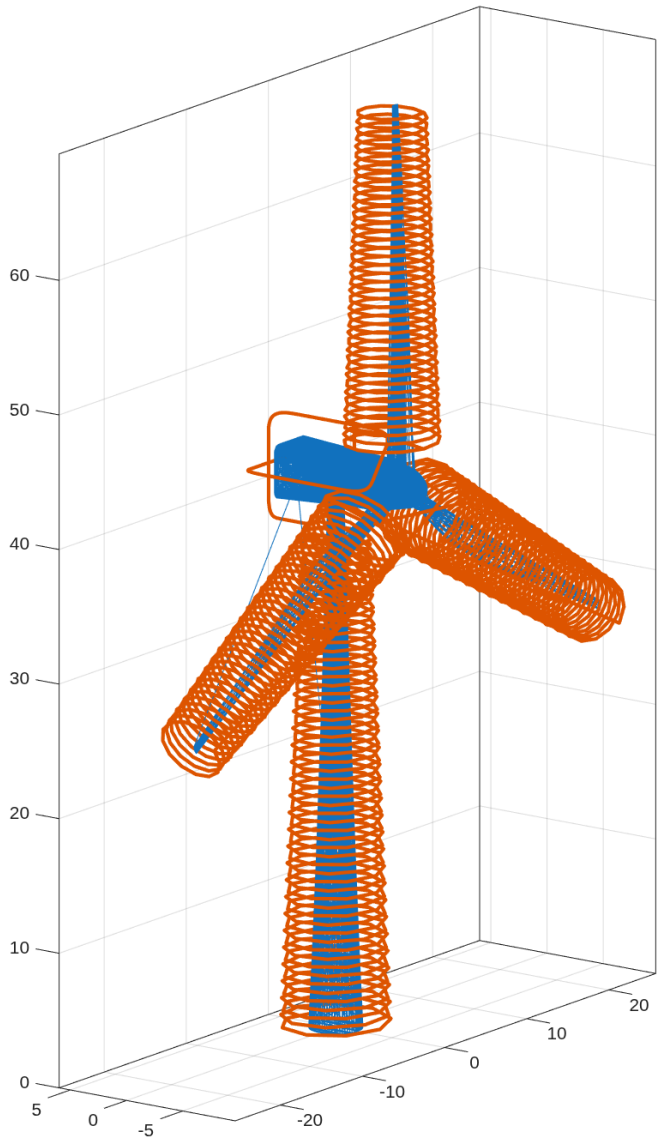


Figure 8: Trajectory plot of drone scanning a wind turbine

6 Conclusion

6.1 Difficulties

For this lab the biggest difficulty was learning how Matlab handles matrices/lists and how that differs from other programming languages. I of course had used matrices previously but then they had been more used purely for mathematical functions. Here having to combine and add elements to them I had to learn more about them.

The second difficulty was the Peter Corke's Robotics Toolbox, mainly how to use the mstraj function. This was solved simply by reading the book and using the web to find how others had used it.

The difficulties I suspected at the lab didn't materialize. Matlab cooperated without any issues and the lab was quite simple math wise.

6.2 Learned

From this lab I've learned:

- How to generate functional trajectories in 3d space using Peter Corke's Robotics Toolbox
- How to combine multiple plots in Matlab
- How to combine, split and append to matrices in Matlab